

REPORT

[인공지능 학기말 프로젝트]



• 과 목 명 : 인 공 지 능

• 담 당 교 수 : 김동근 교수님

• 제 출 일 : 2023. 12. 06.

• 학 과 : 컴퓨터 공학과

• 학 번 : 201601848

• 성 명 : 이 성 남

1. 서론

- 삼목 게임 학습

<https://github.com/datasets/tic-tac-toe> 사이트의 "tic-tac-toe.csv"의 삼목 게임을 tensorflow 로 학습한다.

- 컬러(color) 분류

목적: 카메라로 촬영한 컬러(R, G, B, Yellow, Pink 등) 식별, 색상 종류는 각자 정하기

2. 본론

2.1. 삼목게임 학습

1) 서론 (문제 설명)

➔ 깃허브(<https://github.com/datasets/tic-tac-toe>) 에서 "tic-tac-toe.csv"라는 삼목 게임 데이터를 가져와서 tensorflow 로 학습했습니다. 해당 데이터에서 6 대 4 비율로 훈련 데이터와 테스트 데이터로 나누었습니다. 그리고 훈련 데이터를 통해 모델을 학습시키고 학습시킨 모델을 통해 훈련 데이터와 테스트 데이터를 평가해 보았습니다.

➔ 교재 167p. 2 층신경망: iris 데이터 분류 (step2202) 예제를 활용하여 tic-tac-toe 삼목게임을 학습했습니다.

2) 본론 (구현 내용을 코드, 그래프 등을 사용하여 설명)

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
#1 데이터 읽어오는 함수
# csv파일을 txt파일로 불러온다 txt파일로 불러 오면 ", " 로 구분되어 나누어진다.
# 첫번째 행은 헤더값이 온다. 따라서 skiprows = 1 을 통해 1행을 무시하고 넘어간다.
# converters 를 통해서 문자열을 정수형으로 바꾼다.
def load_iris(shuffle=False):
    label={'true':0, 'false':1, 'x': 2, 'b': 3, 'o': 4}

    data = np.loadtxt(r'C:\Users\wdltjd\Artificial Intelligence\tic-tac-toe.csv', skiprows=1, delimiter=',',
                      converters={i: lambda name: label[name.decode()] for i in range(10)})
    #셔플이 트루라면 데이터를 섞는다.
    if shuffle:
        np.random.shuffle(data)
    return data
```

- ➔ 데이터를 불러와서 csv 파일을 txt 파일로 만들었습니다. 그리고 파일의 첫번째 행에는 데이터 칼럼의 헤더값이 있기 때문에 해당 행을 무시하고 넘어가도록 했습니다. 그리고 칼럼들은 쉼표(,) 로 구분되어 있기 때문에 delimiter = “,” 을 설정하였습니다. 그리고 라벨로 (true, false, x, b, o)를 딕셔너리로 만들어 converters 를 통해서 문자를 정수형으로 바꾸었습니다. 그리고 학습이 잘되게 하기위해 셔플을 할 수 있도록 매개변수로 셔플을 true 로 바꿀 수 있도록 설정하였습니다.

```
# 데이터에서 훈련세트와 테스트세트를 나눈다.
# test_rate -> 테스트 세트 40% iris_data -> 훈련세트 60%
# 80%를 학습하고 20%는 성능을 테스트
def train_test_data_set(iris_data, test_rate=0.2): # train: 0.8, test: 0.2
    n = int(iris_data.shape[0]*(1-test_rate)) # iris_data -> (958, 10) 중 [0] 즉, 958 * 0.8 (766)개 훈련데이터로 설정
    #훈련 데이터 574개
    x_train = iris_data[:n,:-1] # 958의 행중 1행부터 766행까지 사용 + 마지막 열 제거 (입력)
    y_train = iris_data[:n, -1] # 958의 행중 1행부터 766행까지 사용 + 마지막 열만 사용 (출력)
    # 테스트 데이터 384
    x_test = iris_data[n,:-1] # 766행 부터 958행 까지 사용 + 마지막 열 제거 (입력)
    y_test = iris_data[n, -1] # 766행 부터 958행 까지 사용 + 마지막 열만 사용 (출력)
    return (x_train, y_train), (x_test, y_test)
```

- ➔ 데이터를 첫번째 행부터 8 대 2 비율로 나누어 훈련데이터와 테스트 데이터로 나누도록 함수를 만들었습니다. 해당 함수는 데이터를 나누는 비율을 매개변수로 받도록 했습니다. 해당 매개변수는 8 대 2 비율이 고정값으로 되어있습니다.

```
# 데이터 읽어오는 함수를 사용하여 데이터를 불러온다
# 데이터를 섞어서 불러온다 (shuffle = true) -> 데이터를 섞어서 학습이 더 잘된다.
iris_data = load_iris(shuffle=True)
(x_train, y_train), (x_test, y_test) = train_test_data_set(iris_data, test_rate=0.4) #test데이터의 비율을 40%로 높인다.
print("x_train.shape:", x_train.shape) #(574, 4)
print("y_train.shape:", y_train.shape) #(574, 1)
print("x_test.shape:", x_test.shape) #(384, 4)
print("y_test.shape:", y_test.shape) #(384, 1)

# one-hot encoding: 'mse', 'categorical_crossentropy'
y_train = tf.keras.utils.to_categorical(y_train)
y_test = tf.keras.utils.to_categorical(y_test)
##print("y_train=", y_train)
##print("y_test=", y_test)

#2
# 모델을 만든다. n은 뉴런의 갯수, input_dim은 입력갯수, 첫번째 input층 마지막출 output층
# output층은 3가지 종류로 인식할것이기 때문에 unit = 3
n = 30 # number of neurons in a hidden layer
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense(units=n, input_dim=9, activation='relu'))
model.add(tf.keras.layers.Dense(units=2, activation='softmax'))
model.summary()
```

➔ Train_test_data_set() 함수를 사용하여 6 대 4 비율로 훈련데이터와 테스트 데이터를 구분했습니다. 그리고 모델이 나눠진 데이터(y 값)를 구분하게 하기 위해서 원핫 인코딩을 하였습니다. 그리고 뉴런 개수 30 개로 정하고 모델은 Sequential() 모델을 사용하였습니다 그리고 인풋층에 relu 활성화 함수를 사용하고, 아웃풋 층은 softmax 활성화 함수를 사용하였습니다. Unit2 개는 true 와 false 값을 인식할 것이기 때문에 units = 2 를 하였습니다.

```
#3
def MSE(y, t):
    return tf.reduce_mean(tf.square(y - t)) # (y - t)**2

CCE = tf.keras.losses.CategoricalCrossentropy()

opt = tf.keras.optimizers.RMSprop(learning_rate=0.01)
##model.compile(optimizer=opt, loss='mse', metrics=['accuracy'])
##model.compile(optimizer=opt, loss= MSE, metrics=['accuracy'])
##model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])

# model.compile을 통해서 학습환경을 정한다. metrics -> 정확도를 출력
model.compile(optimizer=opt, loss= CCE, metrics=['accuracy'])

# model.fit을 통해 학습하고 값을 ret에 저장
ret = model.fit(x_train, y_train, epochs=1000, batch_size=7, verbose=2) # batch_size=32
```

➔ 손실함수로 원핫인코딩된 값에 적합한 CategoricalCrossentropy()를 사용하였습니다. 그리고 최적화 알고리즘으로 RMSprop 을 사용하였고 학습율을 0.01 로 설정하였습니다. 그리고 정확도 손실율등을 주기적으로 확인하기 위해 compile 함수를 사용하였고, model.fit 을 통해 데이터를 학습했습니다.

```
loss = ret.history['loss']
plt.plot(loss)
plt.xlabel('epochs')
plt.ylabel('loss')
plt.show()
```

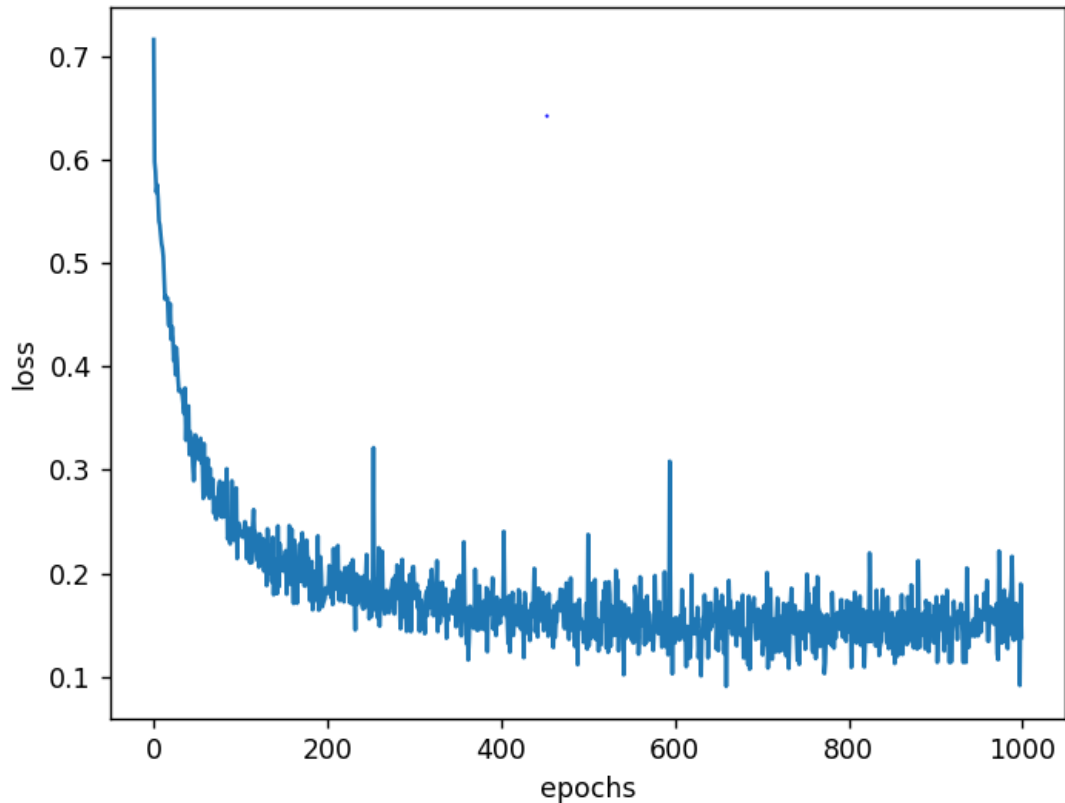
➔ Epoch 당 손실율을 그래프를 통해서 보여줍니다.

```
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=2)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
y_pred = model.predict(x_train)
y_label = np.argmax(y_pred, axis = 1)
C = tf.math.confusion_matrix(np.argmax(y_train, axis = 1), y_label)
print("confusion_matrix(C):", C)
```

➔ 마지막으로 훈련데이터와 테스트데이터를 평가한값을 보여주고 원핫으로 나눠진 데이터결과값을 출력합니다.

3) 결론 (화면 덤프 및 결과 분석 설명)

Figure 1



➔ 모델 학습을 진행하면서(epoch 이 증가하면서) 손실율이 줄어드는 것을 확인 하였습니다.

```
18/18 - 0s - loss: 0.0562 - accuracy: 0.9861 - 133ms/epoch - 7ms/step
12/12 - 0s - loss: 0.2114 - accuracy: 0.9479 - 26ms/epoch - 2ms/step
> 1/18 [>.....] - ETA: 0s
.....> 6/18 [=====>.....] - ETA: 0s
.....> 18/18 [=====] - 0s 4
ms/step
confusion_matrix(0): tf.Tensor(
[[372  3]
 [ 5 194]], shape=(2, 2), dtype=int32)
|
```

➔ 훈련데이터의 정확도와 테스트 데이터의 정확도를 확인했습니다. 각각의 정확도가 0.9861 , 0.9479 임을 확인하였습니다. 해당 데이터의 오차가 생긴이유는 true 값에서 3 개 false 값에서 5 개의 오차가 발생했기 때문임을 알 수 있었습니다.

2.2. 컬러(color) 분류

1) 서론 (문제설명)

- ➔ 저는 의류 이미지를 데이터로 활용하기 위하여 구글을 통해 의류 이미지 데이터셋을 찾아 훈련 데이터로 사용하였습니다. 그리고 테스트 데이터로 제가 직접 찍은 사진을 테스트 데이터로 활용하였습니다. 컬러 구분은 흑색, 회색, 백색을 기준으로 컬러를 구분하였습니다. 해당 컬러는 y 값 (y_test, y_train)으로 사용하였습니다. 그리고 훈련 데이터는 약 1200 장 정도의 의류 이미지를 활용하였고, 테스트 데이터로는 약 35 장의 이미지를 사용하였습니다.
- ➔ 교제 387p. VGG 전이학습(transfer learning): CIFAR-10 분류 (4504) 예제를 활용하여 VGG16 모델을 사용하였습니다. 그리고 교제 539p. COLAB 사용법을 참고하여 COLAB 을 사용하여 모델을 학습하고 학습한 모델로 데이터를 평가했습니다.
- ➔ 추가적으로 최적화 알고리즘으로 RMSprop 와 Adam 을 비교해보았습니다

2) 본론 (구현내용을 코드, 그래프 등을 사용하여 설명)

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.layers import Input, Dense, Flatten
from tensorflow.keras.applications.vgg16 import preprocess_input, VGG16
import numpy as np
import matplotlib.pyplot as plt
import os

#1:
##gpus = tf.config.experimental.list_physical_devices('GPU')
##tf.config.experimental.set_memory_growth(gpus[0], True)

#2

# 이미지 파일들이 있는 디렉토리 경로(xtrain, xtest)
x_test_images_dir = "C:/Users/dltjd/Artificial Intelligence/clothData/test"
x_train_image_dir = "C:/Users/dltjd/Artificial Intelligence/clothData/train"
```

- ➔ 우선적으로 학습을 하기 위한 데이터(이미지)가 저장되어 있는 위치를 각 변수에 저장합니다.

```
# 클래스 레이블 정의
class_labels = {'white': 0, 'gray': 1, 'black': 2}

# 데이터 저장을 위한 리스트 초기화
x_train = []
y_train = []
x_test = []
y_test = []

# 이미지 파일들의 경로를 리스트로 수집
for class_name, class_label in class_labels.items():
    # 훈련 데이터
    train_dir = os.path.join(x_train_image_dir, class_name)
    for filename in os.listdir(train_dir):
        if filename.endswith(".jpg"):
            img_path = os.path.join(train_dir, filename)
            img = load_img(img_path, target_size=(32, 32))
            img_array = img_to_array(img) / 255.0
            x_train.append(img_array)
            y_train.append(class_label)

    # 테스트 데이터
    test_dir = os.path.join(x_test_images_dir, class_name)
    for filename in os.listdir(test_dir):
        if filename.endswith(".jpg"):
            img_path = os.path.join(test_dir, filename)
            img = load_img(img_path, target_size=(32, 32))
            img_array = img_to_array(img) / 255.0
            x_test.append(img_array)
            y_test.append(class_label)

# 리스트를 Numpy 배열로 변환
```

- ➔ Y(train, test) 값을 (백색, 회색, 흑색)으로 딕셔너리를 활용하여 문자열과 해당값에 매칭되도록 밸류값으로 정수형을 선언하였습니다. 그리고 훈련 데이터와 테스트 데이터를 불러와 각각의 이미지에 대해 크기 조정 및 정규화를 진행하도록 하였습니다.

```
# 리스트를 NumPy 배열로 변환
x_train = np.array(x_train)
y_train = np.array(y_train)
x_test = np.array(x_test)
y_test = np.array(y_test)
```

➔ 그리고 for 문을 돌면서 추가된 값들을 numPy 배열로 변환하도록 하였습니다.

```
x_train = x_train.astype('float32') # (사진갯수, 사진높이, 너비, 채널수)
x_test  = x_test.astype('float32')  # (사진갯수, 사진높이, 너비, 채널수)

# one-hot encoding
y_train = tf.keras.utils.to_categorical(y_train)
y_test  = tf.keras.utils.to_categorical(y_test)

# preprocessing, 'caffe', x_train, x_test: BGR
x_train = preprocess_input(x_train)
x_test  = preprocess_input(x_test)
```

➔ 클래스 라벨을 원한 인코딩을 통해서 백색, 흑색, 회색을 구분하도록 하였습니다.
그리고 이미지를 딥러닝 모델에 입력하여 사용하기 위해 preprocess_input() 함수를 사용하여 x_train 과 x_test 를 전처리 하도록 하였습니다.

```
#3: resize_layer
inputs = Input(shape=(32, 32, 3))
resize_layer = tf.keras.layers.Lambda(
    lambda img: tf.image.resize(img,(224, 224)))(inputs)
```

➔ 모델의 입력층을 설정했습니다. 그리고 VGG16 모델에 적합한 이미지 사이즈를 만들기위해 resize_layer 를 만들어서 32, 32 사이즈의 이미지를 224, 224 로 변환합니다.

```
#4:
##W = 'C:/Users/user/.keras/models/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5'
##W = './Data/models/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5'
vgg_model = VGG16(weights='imagenet', include_top=False,
    input_tensor=resize_layer) # input_tensor= inputs
vgg_model.trainable=False
##for layer in vgg_model.layers:
##    layer.trainable = False

#4-1: output: classification
x = vgg_model.output
x = Flatten()(x) # x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
outs = Dense(3, activation='softmax')(x)
model = tf.keras.Model(inputs=inputs, outputs=outs)
model.summary()
```

➔ VGG16 모델을 생성합니다. 그리고 trainable 를 false 로 둬으로써 모델의 가중치를 동결합니다. 그리고 모델의 중간출력을 담고 있는 변수를 만들고 VGG16 의 출력을

x 에 할당합니다. 그리고 Flatten() 을 사용하여 VGG16 의 출력을 1 차원으로 펼칩니다. 그리고 1024 개의 Dense 레이어를 추가하고 활성화 함수로 Relu 를 사용합니다. 추가적으로 출력을 담당하는 Dense 레이어를 추가합니다 해당 과제에서는 흑색, 회색, 백색 3 개의 클래스에대한 확률을 출력하기 때문에 Dense(3, -)으로 하였고 활성화 함수로 softmax 를 사용하였습니다.

그리고 3 개의 클래스에 대한 확률을 출력하는 모델을 생성합니다.

```
#5: train and evaluate the model
filepath = "RES/ckpt/4404-model.h5"
cp_callback = tf.keras.callbacks.ModelCheckpoint(
    filepath, verbose=0, save_best_only=True)
opt = tf.keras.optimizers.RMSprop(learning_rate=0.001)
model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
ret = model.fit(x_train, y_train, epochs=30, batch_size= 64,
    validation_split=0.2, verbose=0, callbacks = [cp_callback])
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=2)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
```

➔ 콜백함수를 사용하여 특정조건에 따라 모델의 체크포인트를 filepath 의 파일 경로로 저장합니다. 그리고 콜백함수의 매개변수로 save_best_only = true 를 설정하여 가장 좋은 체크포인트만 저장하도록 설정하였습니다.

옵티마이저로 RMSprop 를 설정하고 학습율로 0.001 을 설정하였습니다.

(추가적으로 저는 Adam 으로 활성화 알고리즘을 설정하고 학습율 0.001 로 설정해서 실험해보았습니다. 결과는 아래 결과덤프를 통하여 보이도록하겠습니다.)

그리고 원핫 인코딩에 특화된 categorical_crossentropy 손실 함수를 사용하여 컴파일하였고 정확도를 모니터링하도록 컴파일 하였습니다.

그리고 epoch 150 batchsize 30 으로 하여 학습을 진행하였습니다. 과적합을 방지하고 더 좋은 학습 결과를 만들기위해 validation_split=0.2 를 통해 학습데이터의 20 퍼센트는 검증데이터로 사용하도록 하였습니다.

그리고 에폭을 진행할때마다 콜백이 실행되도록 하였습니다.

```
validation_split=0.2, verbose=0, callbacks = [cp_callback],
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=2)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

#6: plot accuracy and loss
fig, ax = plt.subplots(1, 2, figsize=(10, 6))
ax[0].plot(ret.history['loss'], "g-")
ax[0].set_title("train loss")
ax[0].set_xlabel('epochs')
ax[0].set_ylabel('loss')

ax[1].set_ylim(0, 1.1)
ax[1].plot(ret.history['accuracy'], "b-", label="train accuracy")
ax[1].plot(ret.history['val_accuracy'], "r-", label="val accuracy")
ax[1].set_title("accuracy")
ax[1].set_xlabel('epochs')
ax[1].set_ylabel('accuracy')
plt.legend(loc='lower right')
fig.tight_layout()
plt.show()

|
```

➔ 마지막으로 학습된 모델을 통해서 테스트 데이터와 학습 데이터를 평가하였습니다.
그리고 훈련중 발생한 손실율과 정확도를 그래프로 표현하도록 하였습니다.

+ 아담알고리즘 코드

```
x = vgg_model.output
x = Flatten()(x) # x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
outs = Dense(3, activation='softmax')(x)
model = tf.keras.Model(inputs=inputs, outputs=outs)
model.summary()

#5: train and evaluate the model
filepath = "./RES/finalAssignment.h5"
cp_callback = tf.keras.callbacks.ModelCheckpoint(
    filepath, verbose=0, save_best_only=True)
opt = tf.keras.optimizers.Adam(learning_rate=0.001)

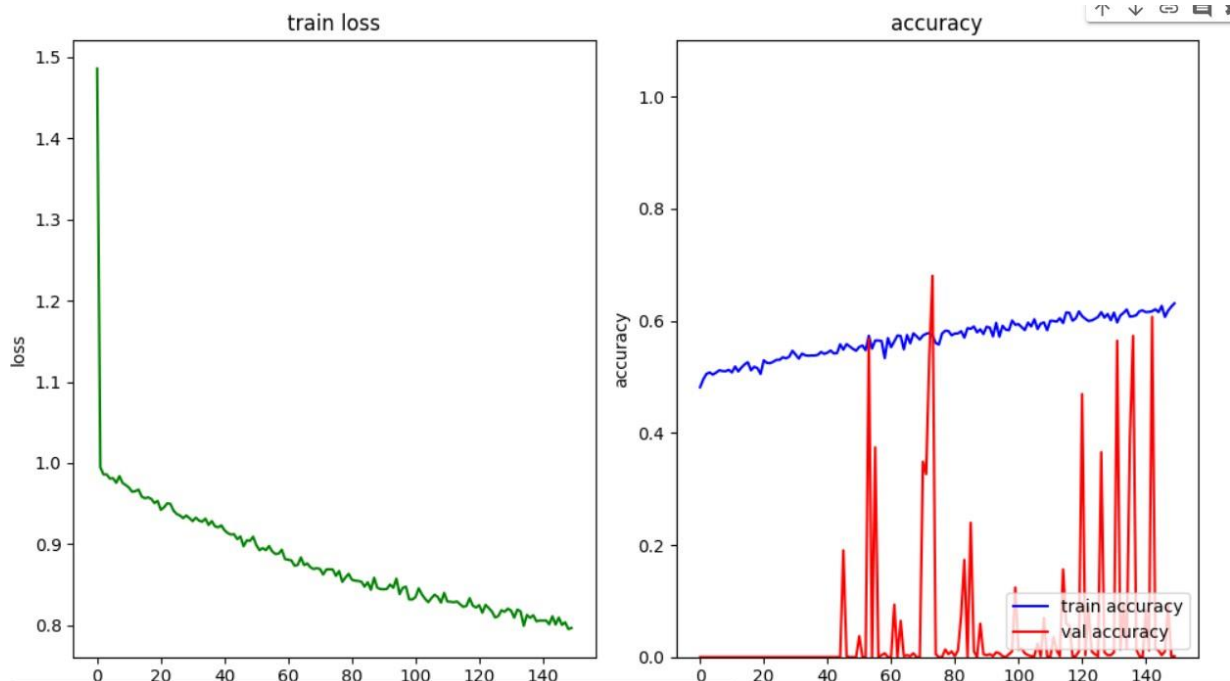
model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
ret = model.fit(x_train, y_train, epochs=120, batch_size= 30,
                validation_split=0.2, verbose=2, callbacks = [cp_callback])
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=2)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
```

3) 결론 (화면 덤프 및 결과 분석 설명)

RMSprop 알고리즘 실행결과

92/92 - 16s - loss: 1.8955 - accuracy: 0.3760 - 16s/epoch - 178ms/step
4/4 - 3s - loss: 1.6864 - accuracy: 0.3651 - 3s/epoch - 865ms/step

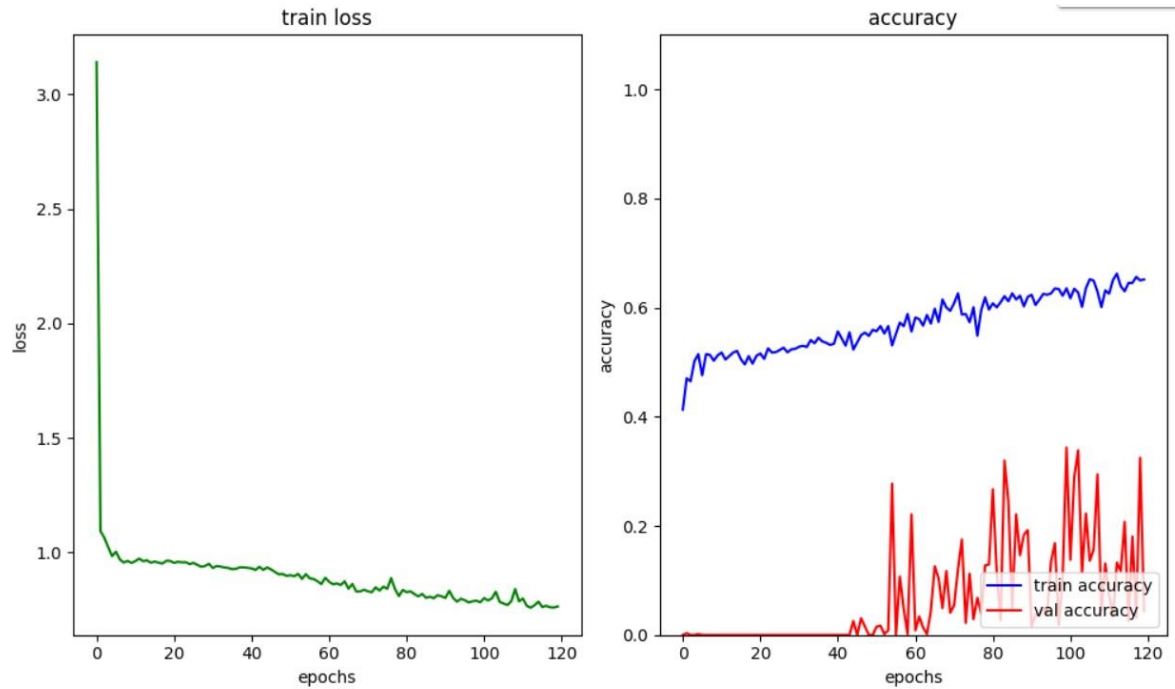
-> 실행결과 정확도가 0.37 로 적게나왔습니다.



➔ Adam 알고리즘 실행결과

92/92 - 15s - loss: 1.2962 - accuracy: 0.5318 - 15s/epoch - 161ms/step
4/4 - 1s - loss: 1.0178 - accuracy: 0.5159 - 568ms/epoch - 142ms/step

아담알고리즘은 에폭을 120 으로 RMSprop 보다 적게 설정하였습니다 . 하지만
정확도가 더 높게 나온 것을 확인할 수있었습니다.



3. 결론(과제 수행 느낌)

➔ 이번 과제를 수행하면서 딥러닝을 직접해보면서 인공지능을 조금 더 알아가는 계기가 되었습니다. 그리고 RMSprop 와 Adam 같은 최적화 알고리즘을 사용해보면서 수업에서 들었던 서로 다른 결과를 도출해낸다는 것을 몸소 느낄 수 있었습니다.

4. 부록(코드)

- 삼목 게임 코드

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

#1 데이터 읽어오는 함수
```

```
# csv 파일을 txt 파일로 불러온다 txt 파일로 불러 오면 "," 로 구분되어 나누어진다.

# 첫번째 행은 헤더값이 온다. 따라서 skiprows = 1 을 통해 1 행을 무시하고 넘어간다.

# converters 를 통해서 문자열을 정수형으로 바꾼다.

def load_Iris(shuffle=False):

    label={'true':0, 'false':1, 'x': 2, 'b': 3, 'o': 4}

    data = np.loadtxt(r'C:\Users\dltjd\Artificial Intelligence\tic-tac-toe.csv', skiprows=1,
delimeter=',',

                    converters={i: lambda name: label[name.decode()] for i in range(10)})

    #셔플이 트루라면 데이터를 섞는다.

    if shuffle:

        np.random.shuffle(data)

    return data

# 데이터에서 훈련세트와 테스트세트를 나눈다.

# test_rate -> 테스트 세트 40% iris_data -> 훈련세트 60%

# 80%를 학습하고 20%는 성능을 테스트

def train_test_data_set(iris_data, test_rate=0.2): # train: 0.8, test: 0.2

    n = int(iris_data.shape[0]*(1-test_rate)) # iris_data -> (958, 10) 중 [0] 즉, 958 * 0.8
(766)개 훈련데이터로 섹

    #훈련 데이터 574 개

    x_train = iris_data[:n,:-1] # 958 의 행중 1 행부터 765 행까지 사용 + 마지막 열 제거
(입력)

    y_train = iris_data[:n, -1] # 958 의 행중 1 행부터 765 행까지 사용 + 마지막 열만 사용
(출력)

    # 테스트 데이터 384
```

```
x_test = iris_data[n,:-1] # 766 행 부터 958 행 까지 사용 + 마지막 열 제거 (입력)

y_test = iris_data[n,-1] # 766 행 부터 958 행 까지 사용 + 마지막 열만 사용 (출력)

return (x_train, y_train), (x_test, y_test)


# 데이터 읽어오는 함수를 사용하여 데이터를 불러온다

# 데이터를 섞어서 불러온다 (shuffle = true) -> 데이터를 섞어야 학습이 더 잘된다.

iris_data = load_Iris(shuffle=True)

(x_train, y_train), (x_test, y_test) = train_test_data_set(iris_data, test_rate=0.4)
#test 데이터의 비율을 40%로 높인다.

print("x_train.shape:", x_train.shape) #(574,4)
print("y_train.shape:", y_train.shape) #(574, 1)
print("x_test.shape:", x_test.shape) #(384, 4)
print("y_test.shape:", y_test.shape) #(384, 1)


# one-hot encoding: 'mse', 'categorical_crossentropy'

y_train = tf.keras.utils.to_categorical(y_train)
y_test = tf.keras.utils.to_categorical(y_test)

##print("y_train=", y_train)
##print("y_test=", y_test)


#2

# 모델을 만든다. n 은 뉴런의 갯수 , input_dim 은 입력갯수, 첫번째 input 층 마지막줄
output 층

# output 층은 3 가지 종류로 인식할것이기 때문에 unit = 3

n = 30 # number of neurons in a hidden layer

model = tf.keras.Sequential()

model.add(tf.keras.layers.Dense(units=n, input_dim=9, activation='relu'))
```

```
model.add(tf.keras.layers.Dense(units=2, activation='softmax'))

model.summary()

#3

def MSE(y, t):

    return tf.reduce_mean(tf.square(y - t)) # (y - t)**2

CCE = tf.keras.losses.CategoricalCrossentropy()

opt = tf.keras.optimizers.RMSprop(learning_rate=0.01)

##model.compile(optimizer=opt, loss='mse', metrics=['accuracy'])

##model.compile(optimizer=opt, loss= MSE, metrics=['accuracy'])

##model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])

# model.compile 을 통해서 학습환경을 정한다. metrics -> 정확도를 출력

model.compile(optimizer=opt, loss= CCE, metrics=['accuracy'])

# model.fit 을 통해 학습하고 값을 ret 에 저장

ret = model.fit(x_train, y_train, epochs=1000, batch_size=7, verbose=2) # batch_size=32

print("len(model.layers):", len(model.layers)) # 2

loss = ret.history['loss']

plt.plot(loss)

plt.xlabel('epochs')

plt.ylabel('loss')

plt.show()

#4
```

```
##print(model.get_weights())

##for i in range(len(model.layers)):

##  print("layer :", i, '-'*20)

##  w = model.layers[i].weights[0].numpy()

##  b = model.layers[i].bias.numpy()

##  print("weights[{}]: {}".format(i, np.array2string(w)))

##  print("bias[{}]: {}".format(i, np.array2string(b)))

train_loss, train_acc = model.evaluate(x_train, y_train, verbose=2)

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

y_pred = model.predict(x_train)

y_label = np.argmax(y_pred, axis = 1)

C = tf.math.confusion_matrix(np.argmax(y_train, axis = 1), y_label)

print("confusion_matrix(C):", C)
```

- 컬러 분류 코드

```
-
- import tensorflow as tf
- from tensorflow.keras.preprocessing.image import load_img,
  img_to_array
- from tensorflow.keras.layers import Input, Dense, Flatten
- from tensorflow.keras.applications.vgg16 import preprocess_input,
  VGG16
- import numpy as np
- import matplotlib.pyplot as plt
- import os
-
- #1:
- ##gpus = tf.config.experimental.list_physical_devices('GPU')
- ##tf.config.experimental.set_memory_growth(gpus[0], True)
-
- #2
-
- # 이미지 파일들이 있는 디렉토리 경로(xtrain, xtest)
- x_test_images_dir = "/content/drive/My Drive/ai/clothData/test"
- x_train_image_dir = "/content/drive/My Drive/ai/clothData/train"
```

```
-
- # 클래스 레이블 정의
- class_labels = {'white': 0, 'gray': 1, 'black': 2}
-
- # 데이터 저장을 위한 리스트 초기화
- x_train = []
- y_train = []
- x_test = []
- y_test = []
-
- # 이미지 파일들의 경로를 리스트로 수집
- for class_name, class_label in class_labels.items():
-     # 훈련 데이터
-     train_dir = os.path.join(x_train_image_dir, class_name)
-     for filename in os.listdir(train_dir):
-         if filename.endswith(".jpg"):
-             img_path = os.path.join(train_dir, filename)
-             img = load_img(img_path, target_size=(32, 32))
-             img_array = img_to_array(img) / 255.0
-             x_train.append(img_array)
-             y_train.append(class_label)
-
-     # 테스트 데이터
-     test_dir = os.path.join(x_test_images_dir, class_name)
-     for filename in os.listdir(test_dir):
-         if filename.endswith(".jpg"):
-             img_path = os.path.join(test_dir, filename)
-             img = load_img(img_path, target_size=(32, 32))
-             img_array = img_to_array(img) / 255.0
-             x_test.append(img_array)
-             y_test.append(class_label)
-
- # 리스트를 NumPy 배열로 변환
- x_train = np.array(x_train)
- y_train = np.array(y_train)
- x_test = np.array(x_test)
- y_test = np.array(y_test)
-
- x_train = x_train.astype('float32') # (사진갯수, 사진높이, 너비,
채널수)
- x_test = x_test.astype('float32') # (사진갯수, 사진높이, 너비,
채널수)
-
```

```
- # one-hot encoding
- y_train = tf.keras.utils.to_categorical(y_train)
- y_test  = tf.keras.utils.to_categorical(y_test)
-
- # preprocessing, 'caffe', x_train, x_test: BGR
- x_train = preprocess_input(x_train)
- x_test  = preprocess_input(x_test)
-
- #3: resize_layer
- inputs = Input(shape=(32, 32, 3))
- resize_layer = tf.keras.layers.Lambda(
-     lambda img: tf.image.resize(img, (224,
- 224))) (inputs)
-
- #4:
- ##W =
- 'C:/Users/user/.keras/models/vgg16_weights_tf_dim_ordering_tf_kern
- els_notop.h5'
- ##W =
- './Data/models/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5'
- vgg_model = VGG16(weights='imagenet', include_top= False,
-     input_tensor= resize_layer) # input_tensor=
- inputs
- vgg_model.trainable=False
- ##for layer in vgg_model.layers:
- ##    layer.trainable = False
-
- #4-1: output: classification
- x = vgg_model.output
- x = Flatten()(x) # x = GlobalAveragePooling2D()(x)
- x = Dense(1024, activation='relu')(x)
- outs = Dense(3, activation='softmax')(x)
- model = tf.keras.Model(inputs=inputs, outputs=outs)
- model.summary()
-
- #5: train and evaluate the model
- filepath = "./RES/finalAssignment.h5"
- cp_callback = tf.keras.callbacks.ModelCheckpoint(
-     filepath, verbose=0, save_best_only=True)
- opt = tf.keras.optimizers.Adam(learning_rate=0.001)
-
- model.compile(optimizer=opt, loss='categorical_crossentropy',
-     metrics=['accuracy'])
- ret = model.fit(x_train, y_train, epochs=120, batch_size= 30,
```

```
-         validation_split=0.2, verbose=2, callbacks =  
    [cp_callback])  
- train_loss, train_acc = model.evaluate(x_train, y_train,  
    verbose=2)  
- test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)  
-  
- #6: plot accuracy and loss  
- fig, ax = plt.subplots(1, 2, figsize=(10, 6))  
- ax[0].plot(ret.history['loss'], "g-")  
- ax[0].set_title("train loss")  
- ax[0].set_xlabel('epochs')  
- ax[0].set_ylabel('loss')  
-  
- ax[1].set_ylim(0, 1.1)  
- ax[1].plot(ret.history['accuracy'], "b-", label="train  
    accuracy")  
- ax[1].plot(ret.history['val_accuracy'], "r-", label="val  
    accuracy")  
- ax[1].set_title("accuracy")  
- ax[1].set_xlabel('epochs')  
- ax[1].set_ylabel('accuracy')  
- plt.legend(loc='lower right')  
- fig.tight_layout()  
- plt.show()  
-  
-
```